

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 5 — Skills

La prima skill

La consegna

L'esercitazione M5-E01: la prima skill, dodici minuti

- Nel Modulo 5, le **skill**: una skill è una cartella con uno `SKILL.md`, un frontmatter YAML e il contenuto in Markdown. Qui ne scrivete **una vera**, la guardate partire nei due modi, e la provate in una sessione nuova
- **La consegna:** una skill di progetto, `riassunto-modifiche`, che riassume le modifiche **non ancora committate** e segnala ciò che è rischioso, su `progetti/urgenze`, un progetto sintetico del corso: la regola con cui l'assistenza di CiaspoLink decide se un guasto è urgente
- **Le frasi alla prova**, dalla pagina «Extend Claude with skills»: «Claude uses skills when relevant, or you can invoke one directly with /skill-name.» E sulle righe con il punto esclamativo: «The command output replaces the placeholder, so Claude receives actual data, not the command itself.»
- La **verifica** dice che il lavoro è fatto, non che la skill è buona. Ciò che Claude sceglie di fare — la chiamata `Skill`, il file cambiato nella risposta, gli strumenti prima di rispondere — lo stampa come un **dato**, non come un esito
- **Il ciclo** di ogni esercitazione: `tools/prepara_esercizio.sh M5-E01` → la sessione in Claude Code → `tools/verifica.sh M5-E01`
- Il README: `m5_skills/s4_skills/m5_e1_la_prima_skill/M5-E01_la_prima_skill/`, nel repository delle esercitazioni

I passi

Dal README: preparare, scrivere la skill, provarla nei due modi

```
# dalla cartella esercitazioni: i passi 1 e 2 del README
tools/prepara_esercizio.sh M5-E01
cd ~/corso-claude-code/M5-E01
bash prepara_modifica.sh
git status --short    # deve mostrare urgenza.py
claude --model sonnet --effort low
```

- `prepara_modifica.sh` fa la piccola modifica su cui lavora la skill, in `progetti/urgenze/urgenza.py`, **senza committarla**
- Claude Code su **Sonnet**, con effort **low**
- Alla prima apertura Claude Code vi chiede se vi **fidate della cartella**: rispondete sì

I passi

Dal README: preparare, scrivere la skill, provarla nei due modi

il prompt del README, il passo 3, a capo per la slide

```
<instructions>
Scrivi la skill di progetto in
.claude/skills/riassunto-modifiche/SKILL.md: riassume le modifiche
non committate e segnala i rischi. Poi esegui da solo questo
comando: python3 -B -I controlla_skill.py
.claude/skills/riassunto-modifiche
</instructions>

<format>
Description in terza persona, con «Da usare quando …». Nel corpo
!`git status --short` e !`git diff HEAD`, passi numerati,
un modello della risposta, un esempio.
</format>
```

- `.claude/` è una cartella protetta: Claude Code vi chiede di **approvare** la scrittura, e può offrirvi «Yes, and allow Claude to edit files in this project's .claude folder for this session». **Leggete il file** prima di approvarlo. Poi chiede di approvare il **comando** del controllo
- Copiate il prompt dal README, non dalla slide. Oppure scrivetela voi, dal modello `riferimenti/template-SKILL.md`: i comandi sono nello stesso passo

I passi

Dal README: preparare, scrivere la skill, provarla nei due modi

```
# nella stessa sessione di Claude Code: il passo 4
/skills                # un pannello: Esc per chiuderlo
/riassunto-modifiche
/skill-doctor          # un altro pannello: Esc anche qui
/exit
# nel terminale, nello spazio di lavoro: i passi 5 e 6
claude plugin validate .claude/skills
bash prova.sh
# di nuovo nella cartella esercitazioni: il passo 7
cd -
tools/verifica.sh M5-E01
```

- **I due modi:** `/riassunto-modifiche` la invocate voi; `bash prova.sh` fa una domanda come la farebbe chiunque, «Che cosa ho cambiato?», in una **sessione nuova**, e mostra il comando prima di eseguirlo
- `/skills` la elenca, e quelle del vostro account stanno sotto «claude.ai sync»; `/skill-doctor` apre la scheda Stats del pannello dei plugin: quanto costa ogni skill e quanto viene usata. **Esc** chiude i due pannelli

Al lavoro

I passi su una pagina, e se qualcosa si rompe: otto minuti

1. `tools/prepara_esercizio.sh M5-E01`, `cd` nello spazio di lavoro, `bash prepara_modifica.sh`
2. `claude --model sonnet --effort low`, e sì alla fiducia nella cartella
3. Il prompt del README, o il modello: **leggete il file** prima di approvarlo, poi approvate il comando del controllo
4. Nella sessione: `/skills`, `/riassunto-modifiche`, `/skill-doctor`, `/exit`. **Esc** chiude i due pannelli
5. Nel terminale: `claude plugin validate .claude/skills`
6. `bash prova.sh`: «Che cosa ho cambiato?», in una sessione nuova
7. `cd -`, `tools/verifica.sh M5-E01`: le ultime due righe sono «Fatto: …» e «VERIFICA OK». L'avviso su `evals/evals.json` non le ferma: le valutazioni sono del laboratorio

Se qualcosa si rompe

- `controlla_skill.py`, il comando del prompt, copia di `tools/controlla_skill.py`, applica le regole del file. Nello standard Agent Skills: `name` uguale alla cartella, minuscole, cifre e trattini; `description` non vuota, con che cosa fa e quando usarla
- `claude plugin validate .claude/skills` è **un passo, non una garanzia**: con Claude Code 2.1.317, nelle prove del corso, ha lasciato passare YAML non valido
- `/skills` non la elenca: il file è `SKILL.md`, in una cartella con il nome della skill. Se `.claude/skills/` non c'era all'avvio della sessione: `/reload-skills`
- La prova risponde senza chiamare la skill: è un dato, non vi boccia. Riscrivete la `description` con le parole di chi chiede, e rilanciate `bash prova.sh`
- Il resto nel README, «Se qualcosa va storto»; e in «Per approfondire» il rimando alla libreria che lo standard indica per validare una skill, `skills-ref`
- L'impronta: Sonnet, effort low, da 3 a 6 turni brevi, nessun subagent, una chiamata di prova; i controlli e la verifica girano sul vostro computer

Che cosa è successo

Il confronto, a voce, e le righe del controllo nella registrazione del corso

- A voce: sulla domanda di `prova.sh`, Claude ha chiamato la skill? Ha nominato il file cambiato?
- Le risposte stanno nelle righe (`dato`) della vostra verifica. «Le righe (dato) dicono che cosa è successo: non decidono.»
- Se la riga della chiamata dice **no**, la verifica non vi boccia: riscrivete la `description` con le parole di chi chiede
- Perché una sessione nuova, nella pagina delle skill di Claude Code: «A fresh session matters because leftover context from authoring the skill will mask gaps in the written instructions.»
- E non basta vederla partire: l'output si misura a parte, ed è il metodo del **laboratorio** di questo modulo

Che cosa è successo

Il confronto, a voce, e le righe del controllo nella registrazione del corso

la verifica di M5-E01: le righe (dato)

```
(dato) la chiamata Skill sulla domanda naturale: sì
(dato) il file cambiato nella risposta: urgenza.py: sì
(dato) gli strumenti, oltre a Skill, prima della risposta: nessuno
(dato) il corpo arrivato a Claude: nessun !`comando` rimasto: sì;
  urgenza.py: sì
[…]
(dato) tutte le prove di prova/: la chiamata Skill riuscita
  in 3 su 3
[…]
(dato) /riassunto-modifiche: chiamate Skill: 0; strumenti:
  nessuno; il file nella risposta: urgenza.py: sì; turni 1; […]
(dato) /skill-doctor: riassunto-modifiche  projectSettings
  ~80          -     0×  never; turni 0
```

Le righe del controllo, a capo per la slide: «[…]» segna un taglio

Registrazione del corso, 3 novembre 2026, Claude Code 2.1.317, Sonnet, effort low

- La prova del docente, in `claude -p`: «3 su 3» è un dato, non una regola
- Il corpo arriva con i dati dentro, «not the command itself». Con `/riassunto-modifiche` nessuna chiamata `Skill`: il corpo entra subito
- `/skill-doctor`, **prima** delle prove: circa 80 nel contesto, la riga dell'elenco; nessun uso

Una skill si prova in una sessione nuova, con le parole che userebbe chiunque.